HW3 readme

Vinisha Bala Dhayanidhi

1. RESULTS (Q4)

lambda1	lambda2	lambda3	Filename	Perplexity
0.05	0.15	0.8	ppl_0.05_0.15_0.8	399.75
0.1	0.1	0.8	ppl_0.1_0.1_0.8	378.62
0.2	0.3	0.5	ppl_0.2_0.3_0.5	237.26
0.2	0.5	0.3	ppl_0.2_0.5_0.3	213.70
0.2	0.7	0.1	ppl_0.2_0.7_0.1	212.34
0.2	0.8	0	ppl_0.2_0.8_0	236.52
1.0	0	0	pp_1.0_0_0	998.67

(The perplexity is computed in log base 10 by the formula N = word_count + sentence_count - oov_count. The OOV words are skipped and not penalized.)

3. DISCUSSION

- **Best and worst:** The best setting was 0.2 / 0.7 / 0.1 (perplexity 212.34), and the worst was the unigram-only model, 1.0 / 0 / 0 (998.67), which is about 4.7 times higher. A unigram model ignores the previous words entirely, so it cannot predict or use even the most basic local patterns. I also noticed that adding any bigram or trigram information lowers the perplexity a lot.

- **Heavy trigram weight hurts:** The two settings with lambda3 = 0.8 (399.75 and 378.62) are far worse than the settings that put more weight on the bigram. The training set has 675,998 distinct trigrams but only about 1 million tokens, so the trigram is usually missing new text: in the output for ppl_0.2_0.5_0.3, 23,578 of the 40,650 scored positions (about 58%) are marked “(unseen ngrams)”, meaning the full trigram (or, for the first word of a sentence, the bigram) was never seen in training. When the trigram was never seen in training, its probability is 0, and the weight assigned to it is lost rather than redistributed since this model has no backoff or discounting. With lambda3 = 0.8, an unseen trigram leaves at most 0.2 of the total weight on the lower-order terms, so the probability of the word shrinks sharply. Note that these two settings also have the smallest unigram weights, so the comparison changes more than one weight at a time.

- **Bigram is the workhorse:** The best three settings all have lambda2 of 0.5 or more. The bigrams carry real context but are much less sparse than trigrams (346,198 bigram types versus 675,998 trigram types), so they are seen enough to give reliable estimates. A little trigram helps, but only a little. If you hold lambda1 at 0.2, perplexity as a function of lambda3 forms a U shape: 236.52 at lambda3 = 0, 212.34 at 0.1, 213.70 at 0.3, and

237.26 at 0.5. A small trigram weight (0.1) improves on none (about 10% lower perplexity), because when a trigram has been seen, it is a strong and specific predictor. But the gain shrinks quickly: 0.1 and 0.3 are nearly tied with a difference of about 0.6%, and at 0.5 the benefit has disappeared (237.26, almost equal to the no-trigram setting at 236.52).

4. CHALLENGES AND OBSERVATIONS

Condor timing: The ngram_count job took about 31 minutes on condor, but its log shows only about 55 seconds of CPU time, and running the same step on the login node took about 5 seconds. So I figured that the delay was cluster/filesystem overhead and not the code. And because of this, since I wasn't sure if my other code would take longer on patas, I drafted the Q4 table using a scratch copy of the LM and then reran all seven settings against the real wsj_sec0_19.lm, which gave me almost thet same perplexities.